

—— // RUBYRS // ——

rubyrs – Language Reference

v0.1.3

LANGUAGE REFERENCE

every builtin · method · class · global

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTOPENSOURCE

Contents

Keyword	3
Kernel	7
Object	11
Comparable	15
Enumerable	16
Class	21
TrueClass/FalseClass	27
Integer	28
Numeric	32
Rational	39
Complex	43
String	46
Symbol	54
Regexp	57
MatchData	58
Array	59
Hash	70
Set	76
Range	81
Struct	84
Exception	86
Enumerator	87
Enumerator::Lazy	89
Enumerator::Yielder	92
Fiber	93
Proc	94
Method	96
Time	97
Date	101

Keyword

def

define a method; body runs on call, returns the last expression

```
def greet; "hi"; end; greet # => "hi"
```

end

close a def/class/module/do/if/case/begin block

```
class A; end # 'end' closes the class body
```

class

open or reopen a class; 'class A < B' sets the superclass

```
class B < Object; end; B.superclass # => Object
```

module

open or reopen a module (namespace / mixin)

```
module M; end; M.class # => Module
```

self

the current receiver object

```
class A; def me; self; end; end; A.new.me.class # => A
```

super

call the same method in the superclass (bare = pass same args)

```
class B<A; def f; super+1; end; end # calls A#f
```

if

conditional; also the 'expr if cond' statement modifier

```
x = 5 if true; x # => 5
```

elsif

additional condition branch inside an if

```
if false then 1 elsif true then 2 end # => 2
```

else

fallback branch of an if/unless/case/begin

```
if false then 1 else 2 end # => 2
```

unless

negated conditional; also the 'expr unless cond' modifier

```
"y" unless false # => "y"
```

case

multi-way branch matched by 'when' via '==='

```
case 2; when 2 then "b"; end # => "b"
```

when

a case branch; matches its value against the subject with '==='

```
case 2; when 1..3 then "in"; end # => "in"
```

while

loop while the condition is truthy; also a statement modifier

```
i=0; i+=1 while i<3; i # => 3
```

until

loop until the condition is truthy; also a statement modifier

```
i=0; i+=1 until i>=3; i # => 3
```

for

iterate 'for x in enumerable' without a new scope

```
for x in [1,2,3]; print x; end # prints 123
```

in

the 'for x in ...' separator and case/in pattern-match clause

```
case [1,2]; in [a,b]; a+b; end # => 3
```

do

open a block ('each do |x| ... end') or a while/for body

```
[1,2].each do |n| print n end # prints 12
```

then

optional separator after an if/when condition

```
if true then 1 else 2 end # => 1
```

yield

invoke the block passed to the current method

```
def f; yield 5; end; f { |x| p x } # prints 5
```

return

return a value from the current method (nil if omitted)

```
def f; return 9; end; f # => 9
```

break

exit the nearest loop/block, optionally with a value

```
[1,2,3].each { |n| break n if n==2 } # => 2
```

next

skip to the next loop/block iteration, optionally with a value

```
[1,2,3].map { |n| next 0 if n==2; n } # => [1, 0, 3]
```

retry

re-run the begin body from a rescue clause

```
a=0; begin; a+=1; raise if a<2; rescue; retry if a<2; end; a # => 2
```

begin

open an exception-handling block (rescue/ensure/else)

```
begin; raise "e"; rescue => e; e.message; end # => "e"
```

rescue

handle a raised exception; also the 'expr rescue fallback' modifier

```
1/0 rescue "safe" # => "safe"
```

ensure

block that always runs whether or not an exception was raised

```
begin; 1; ensure; p "always"; end # prints always
```

and

low-precedence logical AND (short-circuits)

```
(true and false) # => false
```

or

low-precedence logical OR (short-circuits)

```
(nil or 2) # => 2
```

not

low-precedence logical negation

```
(not true)  # => false
```

nil

the sole NilClass instance; the only falsey object besides false

```
nil.nil?  # => true
```

true

the sole TrueClass instance

```
(true && 1)  # => 1
```

false

the sole FalseClass instance (falsey)

```
(false || 1)  # => 1
```

alias

give a method a second name: 'alias new old'

```
class A; def old; 1; end; alias new old; end; A.new.new  # => 1
```

defined?

describe the expression's kind ("method"/"expression"/...) or nil

```
defined?(String)  # => "constant"
```

Kernel

puts

print each arg (arrays recursed) each on its own line

```
puts "hi" # => nil (prints hi)
```

print

print args joined by \$, terminated by \$\ (both nil default)

```
print "hi" # => nil (prints hi)
```

p

print inspect of each arg; return arg/array/nil

```
p 42 # => 42 (also prints 42)
```

pp

alias of p (no pretty-printing)

```
pp [1,2] # => [1, 2] (also prints it)
```

require

no-op: always returns true (no file loaded)

```
require "json" # => true (nothing loaded)
```

require_relative

no-op: always returns true (no file loaded)

```
require_relative "x" # => true (no-op)
```

load

no-op: always returns true (no file loaded)

```
load "x.rb" # => true (no-op)
```

intercept

register AOP before/after/around advice by glob pattern

```
intercept("foo", :before, :log) # => :log
```

raise

raise by class/message/instance (default RuntimeError)

```
raise "boom" rescue "caught" # => "caught"
```

fail

raise by class/message/instance (default RuntimeError)

```
fail "boom" rescue "caught" # => "caught"
```

rand

random Int in [0,n) or Float in [0,1)

```
rand(1) # => 0
```

sleep

no-op: returns 0 without sleeping

```
sleep(5) # => 0 (no actual sleep)
```

srand

reseed PRNG; return the previous seed

```
srand(0) # => previous seed (Int)
```

Integer

convert to Integer, optional base for string args

```
Integer("ff", 16) # => 255
```

Float

convert numeric/string to Float, else raise

```
Float("3.5") # => 3.5
```

Rational

build Rational(num,den); raise on zero denominator

```
Rational(6, 4) # => (3/2)
```

Complex

build Complex from real and imaginary args

```
Complex(1, 2) # => (1+2i)
```

String

convert arg via to_s to a String

```
String(123) # => "123"
```

Array

wrap arg as Array (hash->pairs, nil->[], scalar->[x])

```
Array(nil) # => []
```

format

sprintf-format string; trailing Hash for named refs

```
format("%05.2f", 3.1) # => "03.10"
```

sprintf

sprintf-format string; trailing Hash for named refs

```
sprintf("%d-%d", 1, 2) # => "1-2"
```

gets

read one line from stdin (nil at EOF)

```
line = gets # reads one line from stdin; nil at EOF
```

proc

return the given block as a Proc

```
proc { |x| x+1 }.call(4) # => 5
```

lambda

return block marked as a lambda

```
lambda { |x| x*2 }.call(3) # => 6
```

loop

loop block forever; rescue StopIteration; break value

```
i=0; loop { i+=1; break i if i>2 } # => 3
```

catch

run block with tag; return thrown value if throw matches

```
catch(:x) { throw :x, 42 } # => 42
```

throw

unwind to matching catch(tag) with optional value

```
catch(:t) { throw :t } # => nil
```

block_given?

true if a block is present in the current call

```
def f; block_given?; end; f { } # => true
```

exit

exit process (true/nil->0, false->1, n->n)

```
exit # terminates the process with status 0
```

exit!

exit process (true/nil->0, false->1, n->n)

```
exit!(1) # terminates the process with status 1
```

abort

write optional msg to stderr and exit 1

```
abort("fatal") # writes fatal to stderr, exits 1
```

Object

to_s

host default string of receiver (only when args are empty)

```
5.to_s    # => "5"
```

inspect

host inspect string of receiver

```
[1, 2].inspect    # => "[1, 2]"
```

class

returns the Class object reference, not its name

```
1.class    # => Integer
```

nil?

true only for nil (Value::Undef)

```
nil.nil?    # => true
```

to_json

JSON-encodes any value; ignores optional generator-state arg

```
[1, 2].to_json    # => "[1,2]"
```

==

host structural equality

```
(1 == 1)    # => true
```

!=

negation of ==

```
(1 != 2)    # => true
```

===

case-equality: Class/Regexp/Range/Float-range membership, else ==

```
((1..5) === 3)    # => true
```

is_a?

true if receiver is an instance of the named class/module

```
1.is_a?(Integer) # => true
```

kind_of?

true if receiver is an instance of the named class/module

```
1.kind_of?(Numeric) # => true
```

instance_of?

true if receiver's exact class equals the argument

```
1.instance_of?(Integer) # => true
```

itself

returns the receiver unchanged

```
5.itself # => 5
```

freeze

records receiver as frozen and returns it; not enforced

```
5.freeze # => 5 (frozen flag, not enforced)
```

equal?

object identity: same heap handle or same immediate value

```
1.equal?(1) # => true
```

object_id

stable identity integer (2n+1 for ints, fixed immediates)

```
1.object_id # => 3
```

__id__

stable identity integer (2n+1 for ints, fixed immediates)

```
1.__id__ # => 3
```

dup

shallow copy, never frozen

```
[1, 2].dup # => [1, 2]
```

clone

shallow copy preserving the original's frozen state

```
[1, 2].clone # => [1, 2]
```

frozen?

reports whether receiver is frozen

```
"x".frozen? # => false
```

instance_variable_get

reads ivar by name (@ prefix optional)

```
Object.new.instance_variable_get(:@x) # => nil (unset)
```

instance_variable_set

sets ivar by name and returns the value

```
Object.new.instance_variable_set(:@x, 7) # => 7
```

instance_variables

ivar names as symbols

```
Object.new.instance_variables # => [] (explicit receiver)
```

tap

yields receiver to block, returns receiver

```
5.tap { |x| p x } # => 5 (prints 5)
```

then

passes receiver to block, returns block result (else receiver)

```
5.then { |x| x+1 } # => 6
```

yield_self

passes receiver to block, returns block result (else receiver)

```
5.yield_self { |x| x*2 } # => 10
```

lazy

wraps enumerable in lazy pipeline; non-range/array becomes array

```
[1, 2, 3].lazy.class # => Enumerator::Lazy
```

methods

user-defined instance method names as symbols; builtins omitted

```
Object.new.methods # => [] (builtins omitted)
```

send

dispatches named method with remaining args (no visibility check)

```
5.send(:+, 3) # => 8
```

__send__

dispatches named method with remaining args (no visibility check)

```
(-5).__send__(:abs) # => 5
```

public_send

dispatches named method with remaining args (no visibility check)

```
5.public_send(:+, 1) # => 6
```

method

captures a bound Method object for the named method

```
5.method(:+).call(3) # => 8
```

respond_to?

true if class/respond_to_missing? defines it; builtins permissive

```
"x".respond_to?(:upcase) # => true
```

method_missing

wildcard: forwards any undefined method to user method_missing

```
class F; def method_missing(n,*a);n;end;end;F.new.foo # => :foo
```

Comparable

<

true if <=> is negative; incomparable raises ArgumentError

```
(1 < 2) # => true
```

<=

true if <=> is <= 0; incomparable raises ArgumentError

```
(2 <= 2) # => true
```

>

true if <=> is positive; incomparable raises ArgumentError

```
(3 > 2) # => true
```

>=

true if <=> is >= 0; incomparable raises ArgumentError

```
(2 >= 3) # => false
```

between?

true if lo <= self <= hi via <=>

```
3.between?(1, 5) # => true
```

clamp

confines self to [lo, hi] via <=>

```
3.clamp(4, 10) # => 4
```

Enumerable

map

materialize via each; delegates to Array#map

```
(1..3).map { |x| x*2 } # => [2, 4, 6]
```

collect

materialize via each; delegates to Array#collect

```
(1..3).collect { |x| x*2 } # => [2, 4, 6]
```

flat_map

materialize via each; delegates to Array#flat_map

```
(1..2).flat_map { |x| [x, x] } # => [1, 1, 2, 2]
```

collect_concat

materialize via each; delegates to Array#collect_concat

```
[1,2,3].collect_concat { |x| [x] } # => [1, 2, 3]
```

select

materialize via each; delegates to Array#select

```
(1..5).select(&:even?) # => [2, 4]
```

filter

materialize via each; delegates to Array#filter

```
(1..5).filter(&:odd?) # => [1, 3, 5]
```

filter_map

materialize via each; delegates to Array#filter_map

```
(1..5).filter_map { |x| x*2 if x.even? } # => [4, 8]
```

reject

materialize via each; delegates to Array#reject

```
(1..5).reject(&:even?) # => [1, 3, 5]
```

reduce

materialize via each; delegates to Array#reduce

```
(1..4).reduce(:+) # => 10
```

inject

materialize via each; delegates to Array#inject

```
(1..4).inject(:+) # => 10
```

to_a

materialize via each; delegates to Array#to_a

```
(1..3).to_a # => [1, 2, 3]
```

entries

materialize via each; delegates to Array#entries

```
(1..3).entries # => [1, 2, 3]
```

find

materialize via each; delegates to Array#find

```
(1..5).find(&:even?) # => 2
```

detect

materialize via each; delegates to Array#detect

```
(1..5).detect { |x| x>3 } # => 4
```

find_index

materialize via each; delegates to Array#find_index

```
(1..5).find_index(3) # => 2
```

count

materialize via each; delegates to Array#count

```
(1..5).count # => 5
```

min

materialize via each; delegates to Array#min

```
(1..5).min # => 1
```

max

materialize via each; delegates to Array#max

```
(1..5).max # => 5
```

minmax

materialize via each; delegates to Array#minmax

```
(1..5).minmax # => [1, 5]
```

min_by

materialize via each; delegates to Array#min_by

```
(1..5).min_by { |x| -x } # => 5
```

max_by

materialize via each; delegates to Array#max_by

```
(1..5).max_by { |x| -x } # => 1
```

sort

materialize via each; delegates to Array#sort

```
(1..3).sort # => [1, 2, 3]
```

sort_by

materialize via each; delegates to Array#sort_by

```
(1..3).sort_by { |x| -x } # => [3, 2, 1]
```

sum

materialize via each; delegates to Array#sum

```
(1..3).sum # => 6
```

include?

materialize via each; delegates to Array#include?

```
(1..5).include?(3) # => true
```

member?

materialize via each; delegates to Array#member?

```
(1..5).member?(3) # => true
```

first

materialize via each; delegates to Array#first

```
(1..5).first(2) # => [1, 2]
```

take

materialize via each; delegates to Array#take

```
(1..5).take(2) # => [1, 2]
```

drop

materialize via each; delegates to Array#drop

```
(1..5).drop(2) # => [3, 4, 5]
```

take_while

materialize via each; delegates to Array#take_while

```
(1..5).take_while { |x| x<3 } # => [1, 2]
```

drop_while

materialize via each; delegates to Array#drop_while

```
(1..5).drop_while { |x| x<3 } # => [3, 4, 5]
```

each_with_index

materialize via each; delegates to Array#each_with_index

```
(1..3).each_with_index.to_a # => [[1, 0], [2, 1], [3, 2]]
```

each_with_object

materialize via each; delegates to Array#each_with_object

```
(1..3).each_with_object([]) { |x,a| a << x*2 } # => [2, 4, 6]
```

group_by

materialize via each; delegates to Array#group_by

```
(1..4).group_by(&:even?) # => {false => [1, 3], true => [2, 4]}
```

partition

materialize via each; delegates to Array#partition

```
(1..4).partition(&:even?) # => [[2, 4], [1, 3]]
```

tally

materialize via each; delegates to Array#tally

```
(1..3).tally # => {1 => 1, 2 => 1, 3 => 1}
```

uniq

materialize via each; delegates to Array#uniq

```
[1,1,2].each.uniq # => [1, 2]
```

zip

materialize via each; delegates to Array#zip

```
(1..3).zip([4, 5, 6]) # => [[1, 4], [2, 5], [3, 6]]
```

any?

materialize via each; delegates to Array#any?

```
(1..5).any? { |x| x.even? } # => true
```

all?

materialize via each; delegates to Array#all?

```
(1..5).all? { |x| x.positive? } # => true
```

none?

materialize via each; delegates to Array#none?

```
(1..5).none? { |x| x>9 } # => true
```

one?

materialize via each; delegates to Array#one?

```
[1,2].each.one? { |x| x>1 } # => true
```

each_slice

materialize via each; delegates to Array#each_slice

```
(1..5).each_slice(2) # => [[1, 2], [3, 4], [5]]
```

each_cons

materialize via each; delegates to Array#each_cons

```
(1..4).each_cons(2) # => [[1, 2], [2, 3], [3, 4]]
```

chunk_while

materialize via each; delegates to Array#chunk_while

```
(1..4).chunk_while { |a,b| b-a==1 } # => [[1, 2, 3, 4]]
```

to_h

materialize via each; delegates to Array#to_h

```
(1..3).to_h { |x| [x, x*x] } # => {1 => 1, 2 => 4, 3 => 9}
```

Class

new

Struct.new defines a new struct class; other classes instantiate

```
Struct.new(:x).new(1).x # => 1
```

[]

Struct/Set/Hash/Array [...] class constructor forms

```
Array[1, 2, 3] # => [1, 2, 3]
```

members

Struct class: member names as symbols

```
Struct.new(:a, :b).members # => [:a, :b]
```

yield

Fiber.yield: suspends the running fiber with a value

```
Fiber.new { Fiber.yield 1 }.resume # => 1
```

at

Time.at: time from epoch seconds (UTC)

```
Time.at(0).year # => 1970
```

utc

Time.utc: build UTC time from broken-down fields

```
Time.utc(2020, 1, 1).year # => 2020
```

gm

Time.gm: build UTC time from broken-down fields

```
Time.gm(2020).month # => 1
```

local

Time.local: build UTC time from fields (local tz not modeled)

```
Time.local(2020, 6).month # => 6
```

mktime

Time.mktime: build UTC time from fields (local tz not modeled)

```
Time.mktime(2020).year # => 2020
```

now

Time.now/DateTime.now: current system time

```
Time.now.class # => Time
```

civil

Date/DateTime.civil: date(-time) from y,m,d[,h,m,s]

```
Date.civil(2020, 1, 1).year # => 2020
```

today

Date.today: today's date from the system clock (UTC)

```
Date.today.class # => Date
```

jd

Date/DateTime.jd: from a Julian Day Number

```
Date.jd(2451545).class # => Date
```

parse

Date/DateTime.parse: ISO string, else raises ArgumentError

```
Date.parse("2020-06-15").month # => 6
```

PI

Math::PI constant

```
Math::PI # => 3.141592653589793
```

E

Math::E constant

```
Math::E.class # => Float
```

sqrt

Math.sqrt: square root

```
Math.sqrt(16) # => 4.0
```

cbrt

Math.cbrt: cube root

```
Math.cbrt(27) # => 3.0
```

sin

Math.sin

```
Math.sin(0)    # => 0.0
```

cos

Math.cos

```
Math.cos(0)    # => 1.0
```

tan

Math.tan

```
Math.tan(0)    # => 0.0
```

asin

Math.asin

```
Math.asin(0)   # => 0.0
```

acos

Math.acos

```
Math.acos(1)   # => 0.0
```

atan

Math.atan

```
Math.atan(0)   # => 0.0
```

atan2

Math.atan2(x, y)

```
Math.atan2(1, 1) # => 0.7853981633974483
```

sinh

Math.sinh

```
Math.sinh(0)   # => 0.0
```

cosh

Math.cosh

```
Math.cosh(0)   # => 1.0
```

tanh

Math.tanh

```
Math.tanh(0)   # => 0.0
```

asinh

Math.asinh

```
Math.asinh(0)    # => 0.0
```

acosh

Math.acosh

```
Math.acosh(1)    # => 0.0
```

atanh

Math.atanh

```
Math.atanh(0)    # => 0.0
```

exp

Math.exp: e^{**x}

```
Math.exp(0)      # => 1.0
```

log

Math.log: natural log, or log base y when given 2 args

```
Math.log(8, 2)   # => 3.0
```

log2

Math.log2

```
Math.log2(8)     # => 3.0
```

log10

Math.log10

```
Math.log10(1000) # => 3.0
```

hypot

Math.hypot(x, y)

```
Math.hypot(3, 4) # => 5.0
```

ldexp

Math.ldexp: $x * 2^{**exp}$

```
Math.ldexp(1, 3) # => 8.0
```

gamma

Math.gamma

```
Math.gamma(5)    # => 24.0 (approx)
```

erf

Math.erf

```
Math.erf(1) # => 0.8427 (approx)
```

erfc

Math.erfc: 1 - erf(x)

```
Math.erfc(1) # => 0.1573 (approx)
```

generate

JSON.generate/dump: encode value to JSON string

```
JSON.generate([1, 2]) # => "[1,2]"
```

dump

JSON.generate/dump: encode value to JSON string

```
JSON.dump([1, 2]) # => "[1,2]"
```

pretty_generate

JSON.pretty_generate: indented JSON string

```
JSON.pretty_generate([1]) # => "[\n 1\n]"
```

load

JSON.parse/load: decode JSON; symbolize_names option

```
JSON.load("[1, 2]") # => [1, 2]
```

name

the class name string

```
Integer.name # => "Integer"
```

superclass

direct superclass ref, or nil for BasicObject

```
Integer.superclass # => Numeric
```

ancestors

ancestor chain as class refs

```
Integer.ancestors # => [Integer, Numeric, Comparable, Object, Kernel, BasicObject]
```

instance_methods

instance method names as symbols; visibility not modeled

```
class B; def f; end; end; B.instance_methods # => [:f]
```

public_instance_methods

same as instance_methods; visibility not modeled

```
class B; def f; end; end; B.public_instance_methods # => [:f]
```

method_defined?

true if method defined on class or ancestor

```
class B; def f; end; end; B.method_defined?(:f) # => true
```

public_method_defined?

same as method_defined?; visibility not modeled

```
class B; def f; end; end; B.public_method_defined?(:f) # => true
```

INFINITY

Float::INFINITY

```
Float::INFINITY # => Infinity
```

NAN

Float::NAN

```
Float::NAN # => NaN
```

MAX

Float::MAX

```
Float::MAX.class # => Float
```

MIN

Float::MIN (smallest positive normal)

```
Float::MIN.class # => Float
```

EPSILON

Float::EPSILON

```
Float::EPSILON.class # => Float
```

DIG

Float::DIG, returns 15

```
Float::DIG # => 15
```

MANT_DIG

Float::MANT_DIG, returns 53

```
Float::MANT_DIG # => 53
```

TrueClass/FalseClass

&

logical AND by truthiness

```
(true & false) # => false
```

|

logical OR by truthiness

```
(true | false) # => true
```

^

logical XOR by truthiness

```
(true ^ true) # => false
```

!

logical negation of receiver truthiness

```
(!true) # => false
```

to_s

host to_s string of true/false/nil

```
true.to_s # => "true"
```

inspect

host to_s string of true/false/nil (same as to_s)

```
false.inspect # => "false"
```

to_a

nil.to_a returns [] (guarded to nil only)

```
nil.to_a # => []
```

to_h

nil.to_h returns {} (guarded to nil only)

```
nil.to_h # => {}
```

Integer

to_s

renders bigint in given radix (default 10)

```
255.to_s(16) # => "ff"
```

inspect

renders bigint in given radix (default 10)

```
255.inspect # => "255"
```

to_i

returns self unchanged

```
5.to_i # => 5
```

to_int

returns self unchanged

```
5.to_int # => 5
```

floor

returns self unchanged (ignores any ndigits arg)

```
5.floor(2) # => 5
```

ceil

returns self unchanged (ignores any ndigits arg)

```
5.ceil(2) # => 5
```

round

returns self unchanged (ignores any ndigits arg)

```
5.round(2) # => 5
```

truncate

returns self unchanged (ignores any ndigits arg)

```
5.truncate(2) # => 5
```

to_f

converts to Float, INFINITY when out of f64 range

```
5.to_f    # => 5.0
```

abs

absolute value

```
(-5).abs   # => 5
```

magnitude

absolute value

```
(-5).magnitude # => 5
```

-@

arithmetic negation

```
-(5)      # => -5
```

bit_length

number of bits in the value

```
255.bit_length # => 8
```

even?

true if divisible by 2

```
4.even?      # => true
```

odd?

true if not divisible by 2

```
3.odd?       # => true
```

zero?

true if the value is 0

```
0.zero?      # => true
```

positive?

true if greater than 0

```
5.positive?  # => true
```

negative?

true if less than 0

```
(-5).negative? # => true
```

integer?

always true

```
5.integer?  # => true
```

hash

i64 value, or bit_length as fallback when out of range

```
(10**30).hash  # => 100
```

succ

returns self + 1

```
5.succ  # => 6
```

next

returns self + 1

```
5.next  # => 6
```

pred

returns self - 1

```
5.pred  # => 4
```

digits

digits of abs(self) in base (default 10, min 2), low-first

```
123.digits  # => [3, 2, 1]
```

/

floored division; ZeroDivisionError on 0, nil if not Integer

```
(-7 / 2)  # => -4
```

div

floored division; ZeroDivisionError on 0, nil if not Integer

```
(10**20).div(3)  # => 3333333333333333333
```

%

floored modulo; ZeroDivisionError on 0, nil if not Integer

```
(-7 % 3)  # => 2
```

modulo

floored modulo; ZeroDivisionError on 0, nil if not Integer

```
(-7).modulo(3)  # => 2
```

divmod

[floored quotient, floored modulo]; ZeroDivisionError on 0

```
(-7).divmod(3)    # => [-3, 2]
```

<=>

compare returning -1/0/1, nil when arg is not Integer

```
(1 <=> 2)    # => -1
```

coerce

returns [arg, self] without numeric conversion

```
1.coerce(2)    # => [2, 1]
```

Numeric

times

yields 0...n; block-less returns an Enumerator

```
3.times { |i| print i } # prints 012
```

**

exact Integer power (BigInt on overflow), else Float powf

```
(2 ** 10) # => 1024
```

pow

power; 2-arg form is modular exp, neg exp+mod raises RangeError

```
5.pow(3, 7) # => 6
```

/

Int/Int floored div, Int/Rational exact, else Float; ZeroDivisionError

```
(7 / 2) # => 3
```

%

Int floored mod, else float $x - \text{floor}(x/y)*y$; ZeroDivisionError

```
(-7 % 3) # => 2
```

modulo

Int floored mod, else float $x - \text{floor}(x/y)*y$; ZeroDivisionError

```
7.modulo(3) # => 1
```

step

iterates by step; all-Int stays Int else Float; block-less Enumerator

```
1.step(5, 2) { |i| print i } # prints 135
```

upto

iterates self..limit by 1; block-less returns Enumerator

```
1.upto(3) { |i| print i } # prints 123
```

downto

iterates self down to limit by -1; block-less Enumerator

```
3.downto(1) { |i| print i } # prints 321
```

to_s

Integer in radix 2..=36, else default string form

```
255.to_s(16) # => "ff"
```

to_i

truncates to Integer

```
3.7.to_i # => 3
```

to_int

truncates to Integer

```
3.7.to_int # => 3
```

to_f

converts to Float

```
3.to_f # => 3.0
```

to_r

Int as n/1, Float as exact rational; FloatDomainError on NaN/Inf

```
3.to_r # => (3/1)
```

rationalize

simplest rational within eps (default round-trips the f64)

```
0.3.rationalize # => (3/10)
```

to_c

returns Complex (self + 0i)

```
5.to_c # => (5+0i)
```

coerce

[b, a] as Int when both Int, else both as Float

```
1.5.coerce(2) # => [2.0, 1.5]
```

abs

absolute value, preserving Int/Float type

```
(-5).abs # => 5
```

magnitude

absolute value, preserving Int/Float type

```
(-2.5).magnitude    # => 2.5
```

abs2

self * self, preserving Int/Float type

```
3.abs2             # => 9
```

even?

true if as_i(self) % 2 == 0

```
4.even?            # => true
```

odd?

true if as_i(self) % 2 != 0

```
3.odd?             # => true
```

zero?

true if value equals 0.0

```
0.0.zero?          # => true
```

nonzero?

returns self if non-zero, else nil

```
5.0.nonzero?       # => 5.0
```

positive?

true if greater than 0

```
5.positive?        # => true
```

negative?

true if less than 0

```
(-5).negative?     # => true
```

integer?

true for Int, false for Float

```
3.5.integer?       # => false
```

succ

returns as_i(self) + 1

```
5.succ             # => 6
```

next

returns as_i(self) + 1

```
5.next    # => 6
```

pred

returns as_i(self) - 1

```
5.pred    # => 4
```

floor

floor to optional ndigits; Float only when ndigits > 0

```
3.14159.floor(2)    # => 3.14
```

ceil

ceil to optional ndigits; Float only when ndigits > 0

```
3.7.ceil    # => 4
```

round

round to optional ndigits; Float only when ndigits > 0

```
3.14159.round(2)    # => 3.14
```

truncate

truncate to optional ndigits; Float only when ndigits > 0

```
3.7.truncate    # => 3
```

nan?

true if Float is NaN (only dispatched for Float receiver)

```
(0.0/0.0).nan?    # => true
```

infinite?

1/-1 if infinite Float, else nil

```
(1.0/0.0).infinite?    # => 1
```

finite?

true if finite Float, true for Int

```
5.0.finite?    # => true
```

divmod

Int [floor div, floor mod] else [floor q as Int, Float rem]

```
7.divmod(2)    # => [3, 1]
```

chr

1-char string from low byte of self (u8 as char)

```
65.chr    # => "A"
```

gcd

gcd(self, arg); ArgumentError if no arg

```
10.gcd(4)  # => 2
```

lcm

lcm(self, arg), 0 if either is 0; ArgumentError if no arg

```
4.lcm(6)   # => 12
```

gcdlcm

[gcd, lcm]; ArgumentError if no arg

```
6.gcdlcm(4) # => [2, 12]
```

ceildiv

ceiling division -floor_div(-a,b); ZeroDivisionError

```
10.ceildiv(3) # => 4
```

[]

bit i of two's-complement; neg i -> 0, i>=64 -> sign bit

```
5[0]    # => 1
```

digits

digits in base (default 10, min 2); Math::DomainError if negative

```
123.digits # => [3, 2, 1]
```

bit_length

bit length of self, using ~n for negatives

```
255.bit_length # => 8
```

fdiv

float division of self by arg

```
5.fdiv(2)    # => 2.5
```

clamp

clamps to lo/hi or range bounds, returning bound's own type

```
3.clamp(4, 10) # => 4
```

<=>

compare as floats: -1/0/1, nil when unordered (NaN)

```
(5.0 <=> 3.0)  # => 1
```

<<

left shift via BigInt (promotes on overflow); TypeError if non-Int

```
(1 << 4)  # => 16
```

>>

right shift via BigInt; TypeError if arg not Integer

```
(16 >> 2)  # => 4
```

&

bitwise AND via BigInt; TypeError if arg not Integer

```
(6 & 3)  # => 2
```

|

bitwise OR via BigInt; TypeError if arg not Integer

```
(5 | 2)  # => 7
```

^

bitwise XOR via BigInt; TypeError if arg not Integer

```
(5 ^ 1)  # => 4
```

between?

true if arg0 <= self <= arg1 (compared as floats)

```
3.between?(1, 5)  # => true
```

+

addition, Int/Int stays Int else Float

```
(2 + 3)  # => 5
```

-

subtraction, Int/Int stays Int else Float

```
(5 - 3)  # => 2
```

multiplication, Int/Int stays Int else Float

```
(4 * 3)  # => 12
```

<

less-than comparison

```
(1 < 2) # => true
```

>

greater-than comparison

```
(3 > 2) # => true
```

<=

less-or-equal comparison

```
(2 <= 2) # => true
```

>=

greater-or-equal comparison

```
(3 >= 2) # => true
```

Rational

numerator

Returns the numerator as a bigint.

```
Rational(3, 4).numerator # => 3
```

denominator

Returns the denominator as a bigint.

```
Rational(3, 4).denominator # => 4
```

to_f

Converts to Float (NaN if unrepresentable).

```
Rational(1, 2).to_f # => 0.5
```

to_i

Truncates toward zero to an integer.

```
Rational(7, 2).to_i # => 3
```

to_int

Truncates toward zero to an integer.

```
Rational(7, 2).to_int # => 3
```

truncate

Truncates toward zero to an integer (ignores digits arg).

```
Rational(7, 2).truncate # => 3
```

to_r

Returns self unchanged.

```
Rational(1, 2).to_r # => (1/2)
```

abs

Absolute value as a Rational.

```
Rational(-1, 2).abs # => (1/2)
```

magnitude

Absolute value as a Rational.

```
Rational(-1, 2).magnitude # => (1/2)
```

-@

Negates the rational.

```
Rational(0, 1) - Rational(1, 2) # => (-1/2)
```

+@

Returns self unchanged.

```
+Rational(1, 2) # => (1/2)
```

zero?

True if the value is zero.

```
Rational(0, 1).zero? # => true
```

positive?

True if the value is positive.

```
Rational(1, 2).positive? # => true
```

negative?

True if the value is negative.

```
Rational(-1, 2).negative? # => true
```

/

Divides; ZeroDivisionError on 0, Float if arg not rational.

```
(Rational(1, 2) / Rational(1, 4)) # => (2/1)
```

quo

Divides; ZeroDivisionError on 0, Float if arg not rational.

```
Rational(1, 2).quo(Rational(1, 4)) # => (2/1)
```

**

Power; integer exp exact (Rational), else Float via powf.

```
(Rational(2, 3) ** 2) # => (4/9)
```

pow

Power; integer exp exact (Rational), else Float via powf.

```
Rational(2, 3).pow(2) # => (4/9)
```

<=>

Compares; returns -1/0/1 or nil if arg not rational.

```
(Rational(1, 2) <=> Rational(1, 3)) # => 1
```

coerce

Returns [other-as-rational, self].

```
Rational(1, 2).coerce(2) # => [(2/1), (1/2)]
```

hash

Hash as the value's i64 (0 on overflow).

```
Rational(1, 2).hash # => 0
```

integer?

True if the denominator is 1.

```
Rational(4, 2).integer? # => true
```

+

Addition via numeric hook.

```
(Rational(1, 2) + Rational(1, 3)) # => (5/6)
```

-

Subtraction via numeric hook.

```
(Rational(1, 2) - Rational(1, 3)) # => (1/6)
```

Multiplication via numeric hook.

```
(Rational(2, 3) * Rational(3, 4)) # => (1/2)
```

==

Equality via numeric hook.

```
(Rational(1, 2) == Rational(1, 2)) # => true
```

<

Less-than via numeric hook.

```
(Rational(1, 3) < Rational(1, 2)) # => true
```

>

Greater-than via numeric hook.

```
(Rational(1, 2) > Rational(1, 3)) # => true
```

`<=`

Less-or-equal via numeric hook.

```
(Rational(1, 2) <= Rational(1, 2)) # => true
```

`>=`

Greater-or-equal via numeric hook.

```
(Rational(1, 2) >= Rational(1, 3)) # => true
```

Complex

real

Returns the real part.

```
Complex(3, 4).real    # => 3
```

imaginary

Returns the imaginary part.

```
Complex(3, 4).imaginary    # => 4
```

imag

Returns the imaginary part.

```
Complex(3, 4).imag    # => 4
```

to_s

String form via host complex_to_s.

```
Complex(1, 2).to_s    # => "1+2i"
```

abs

Magnitude $\sqrt{\text{re}^2 + \text{im}^2}$ as a Float.

```
Complex(3, 4).abs    # => 5.0
```

magnitude

Magnitude $\sqrt{\text{re}^2 + \text{im}^2}$ as a Float.

```
Complex(3, 4).magnitude    # => 5.0
```

abs2

Squared magnitude $\text{re}^2 + \text{im}^2$.

```
Complex(3, 4).abs2    # => 25
```

conjugate

Complex conjugate (negates imaginary part).

```
Complex(3, 4).conjugate    # => (3-4i)
```

conj

Complex conjugate (negates imaginary part).

```
Complex(3, 4).conj    # => (3-4i)
```

rectangular

Returns [real, imaginary] array.

```
Complex(3, 4).rectangular    # => [3, 4]
```

rect

Returns [real, imaginary] array.

```
Complex(3, 4).rect    # => [3, 4]
```

-@

Negates both real and imaginary parts.

```
Complex(0, 0) - Complex(3, 4)    # => (-3-4i)
```

+@

Returns self unchanged.

```
+Complex(3, 4)    # => (3+4i)
```

==

Equality via host eq_values.

```
(Complex(1, 2) == Complex(1, 2))    # => true
```

+

Addition via numeric hook.

```
(Complex(1, 2) + Complex(3, 4))    # => (4+6i)
```

-

Subtraction via numeric hook.

```
(Complex(5, 6) - Complex(1, 2))    # => (4+4i)
```

*

Multiplication via numeric hook.

```
(Complex(1, 2) * Complex(1, 2))    # => (-3+4i)
```

**

Power by repeated mul; raises on non-integer/negative exp.

```
(Complex(1, 2) ** 2)    # => (-3+4i)
```

pow

Power by repeated mul; raises on non-integer/negative exp.

```
Complex(1, 2).pow(2)  # => (-3+4i)
```

String

length

char count of the string (alias size)

```
"héllö".length # => 5
```

size

char count of the string (alias length)

```
"abc".size # => 3
```

upcase

uppercase; :ascii option limits to ASCII letters

```
"abc".upcase # => "ABC"
```

downcase

lowercase; :ascii option limits to ASCII letters

```
"ABC".downcase # => "abc"
```

swapcase

invert case of each char, returning a new String

```
"Abc".swapcase # => "aBc"
```

capitalize

uppercase first char, lowercase the rest

```
"hELLO".capitalize # => "Hello"
```

reverse

reverse the characters

```
"abc".reverse # => "cba"
```

strip

trim leading and trailing whitespace

```
"  hi  ".strip # => "hi"
```

rstrip

trim leading whitespace

```
" hi".rstrip # => "hi"
```

rstrip

trim trailing whitespace

```
"hi ".rstrip # => "hi"
```

chomp

remove trailing separator/newline; empty arg strips all trailing

```
"hi\n".chomp # => "hi"
```

chop

remove last char; trailing counts as one

```
"hi!".chop # => "hi"
```

chars

array of single-char strings

```
"abc".chars # => ["a", "b", "c"]
```

bytes

array of UTF-8 byte integers

```
"AB".bytes # => [65, 66]
```

bytesize

number of UTF-8 bytes, not chars

```
"é".bytesize # => 2
```

each_byte

with block yield each byte, return self; else Enumerator of bytes

```
"AB".each_byte { |b| } # => "AB"
```

getbyte

byte at index (negatives ok); nil if out of range

```
"AB".getbyte(0) # => 65
```

b

copy of the string (ASCII-8BIT shim, bytes unchanged)

```
"abc".b # => "abc"
```

ascii_only?

true when every byte is 7-bit ASCII

```
"abc".ascii_only? # => true
```

valid_encoding?

always true (only valid UTF-8 is carried)

```
"abc".valid_encoding? # => true
```

force_encoding

no-op shim returning self

```
"abc".force_encoding("UTF-8") # => "abc"
```

encode

no-op shim returning a copy

```
"abc".encode("UTF-8") # => "abc"
```

lines

array of lines (keeping newlines)

```
"a\nb\n".lines # => ["a\n", "b\n"]
```

each_line

with block yield each line, return self; else Enumerator of lines

```
"a\nb\n".each_line { |l| } # => "a\nb\n"
```

center

center-pad to width using pad string (default space)

```
"hi".center(6) # => " hi "
```

tr

translate chars from spec to spec, ranges/negation supported

```
"hello".tr("el","ip") # => "hippo"
```

delete

remove chars matching the spec(s)

```
"hello".delete("l") # => "heo"
```

count

count chars matching the spec(s)

```
"hello".count("l") # => 2
```

squeeze

collapse adjacent duplicate chars (limited to spec if given)

```
"aaabbb".squeeze # => "ab"
```

empty?

true if the string has no chars

```
".empty? # => true
```

to_i

parse leading integer in base (default 10); 0 if none

```
"42abc".to_i # => 42
```

hex

parse leading hex integer; 0 if none

```
"ff".hex # => 255
```

oct

parse leading base-8 int; radix prefix auto-detects base

```
"0x1f".oct # => 31
```

to_f

parse leading float, 0.0 if none

```
"3.14xy".to_f # => 3.14
```

to_r

parse leading rational ("3/4", "3.14"); else 0/1

```
"3/4".to_r # => (3/4)
```

to_s

returns self (alias to_str)

```
"abc".to_s # => "abc"
```

to_str

returns self (alias to_s)

```
"abc".to_str # => "abc"
```

to_sym

the Symbol with this name

```
"abc".to_sym # => :abc
```

include?

true if it contains the substring arg

```
"hello".include?("ell") # => true
```

start_with?

true if any arg matches at the start (Regexp anchored at 0)

```
"hello".start_with?("he") # => true
```

end_with?

true if any string arg is a suffix

```
"hello".end_with?("lo") # => true
```

match?

true if the Regexp/pattern matches; sets no \$~

```
"hello".match?(/l+/) # => true
```

=~

match Regexp, set \$~/ \$1..; return char offset or nil

```
"hello" =~ /l/ # => 2
```

match

return MatchData for the Regexp, else nil

```
"hello".match(/l/) # => #<MatchData "l">
```

scan

with block yield each match return self; else array of matches

```
"a1b2".scan(/\d/) # => ["1", "2"]
```

split

split by pattern/string/awk-mode with optional limit

```
"a,b,c".split(",") # => ["a", "b", "c"]
```

sub

replace first match of Regexp/string via arg or block

```
"hello".sub("l","L") # => "heLlo"
```

gsub

replace all matches of Regexp/string via arg or block

```
"hello".gsub("l","L") # => "heLLo"
```

replace

overwrite receiver contents in place, return self

```
"x".replace("yz") # => "yz"
```

<<

append arg's to_s in place, return self

```
"ab" << "c" # => "abc"
```

+

return new string concatenating self and arg's to_s

```
"ab" + "cd" # => "abcd"
```

concat

append all args in order in place, return self

```
"a".concat("b", "c") # => "abc"
```

<=>

compare strings -1/0/1; nil if arg not a String

```
"a" <=> "b" # => -1
```

between?

true if self is between lo and hi inclusive

```
"b".between?("a", "c") # => true
```

clamp

clamp to lo/hi or inclusive range; exclusive range errors

```
"z".clamp("a", "m") # => "m"
```

*

repeat the string n times

```
"ab" * 3 # => "ababab"
```

%

sprintf-format self with array/hash/single operand

```
"%d-%s" % [1, "x"] # => "1-x"
```

ljust

left-justify padded to width (default space)

```
"hi".ljust(5) # => "hi  "
```

rjust

right-justify padded to width (default space)

```
"hi".rjust(5) # => "   hi"
```

each_char

with block yield each char, return self; else Enumerator of chars

```
"ab".each_char { |c| } # => "ab"
```

[]

substring by index/range/regexp (alias slice)

```
"hello"[1,3] # => "ell"
```

slice

substring by index/range/regexp (alias [])

```
"hello".slice(0,2) # => "he"
```

eq?

content equality, only another String can be equal

```
"ab".eq?("ab") # => true
```

slice!

remove and return the sliced portion in place; nil if none

```
"hello".slice!(0,2) # => "he"
```

ord

codepoint of first char; ArgumentError if empty

```
"A".ord # => 65
```

chr

first char as a string, empty if none

```
"abc".chr # => "a"
```

partition

split on first match of sep into [before, sep, after]

```
"a-b-c".partition("-") # => ["a", "-", "b-c"]
```

rpartition

split on last match of sep into [before, sep, after]

```
"a-b-c".rpartition("-") # => ["a-b", "-", "c"]
```

casecmp

case-insensitive compare returning -1/0/1

```
"ABC".casecmp("abc") # => 0
```

casecmp?

case-insensitive equality boolean

```
"ABC".casecmp?("abc") # => true
```

tr_s

translate like tr then squeeze runs of translated chars

```
"aabbcc".tr_s("ab","x") # => "xccc"
```

succ

next string in succession (alias next)

```
"az".succ # => "ba"
```

next

next string in succession (alias succ)

```
"az".next # => "ba"
```

insert

insert arg before index (negative inserts after) in place

```
"abc".insert(1,"X") # => "aXbc"
```

prepend

prepend all args in place, return self

```
"b".prepend("a") # => "ab"
```

index

char offset of first substring match from optional pos; nil

```
"hello".index("l") # => 2
```

rindex

char offset of last substring match up to optional pos; nil

```
"hello".rindex("l") # => 3
```

[]=

assign to indexed/range/regexp slice in place

```
s="hi"; s[0]="H"; s # => "Hi"
```

Symbol

to_s

the name as a String (alias id2name, name)

```
:abc.to_s    # => "abc"
```

id2name

the name as a String (alias to_s, name)

```
:abc.id2name # => "abc"
```

name

the name as a String (alias to_s, id2name)

```
:abc.name    # => "abc"
```

to_sym

returns self (alias intern)

```
:abc.to_sym  # => :abc
```

intern

returns self (alias to_sym)

```
:abc.intern  # => :abc
```

length

char count of the name (alias size)

```
:abc.length  # => 3
```

size

char count of the name (alias length)

```
:abc.size    # => 3
```

empty?

true if the name is empty

```
".to_sym.empty? # => true
```

upcase

uppercased name as a Symbol

```
:abc.upcase # => :ABC
```

downcase

lowercased name as a Symbol

```
:ABC.downcase # => :abc
```

succ

next name in succession as a Symbol (alias next)

```
:ab.succ # => :ac
```

next

next name in succession as a Symbol (alias succ)

```
:ab.next # => :ac
```

swapcase

case-inverted name as a Symbol

```
:Abc.swapcase # => :aBC
```

capitalize

capitalized name as a Symbol

```
:hello.capitalize # => :Hello
```

[]

index the name, returning a String (alias slice)

```
:hello[1,3] # => "ell"
```

slice

index the name, returning a String (alias [])

```
:hello.slice(0,2) # => "he"
```

start_with?

true if any arg is a prefix of the name

```
:hello.start_with?("he") # => true
```

end_with?

true if any arg is a suffix of the name

```
:hello.end_with?("lo") # => true
```

match?

true if the name matches the Regexp/pattern, no \$~

```
:hello.match?(/l+/) # => true
```

<=>

compare names -1/0/1; nil if arg not a Symbol

```
:a <=> :b # => -1
```

to_proc

proc that sends the named method to its first arg

```
[1,2].map(&:to_s) # => ["1", "2"]
```

Regexp

source

the pattern source string

```
/ab+/.source # => "ab+"
```

match?

true if the pattern matches the arg, no \$~

```
/l+/.match?("hello") # => true
```

=~

match arg, set \$~/ \$1..; return char offset or nil

```
/l/ =~ "hello" # => 2
```

match

return MatchData for the arg string

```
/l/.match("hello") # => #<MatchData "l">
```

scan

array of all matches in the arg string

```
/\d/.scan("a1b2") # => ["1", "2"]
```

to_s

string form "(?-mix:source)"

```
/ab/.to_s # => "(?-mix:ab)"
```

inspect

literal form "/source/"

```
/ab/.inspect # => "/ab/"
```

MatchData

[]

group string at integer index; nil if absent

```
"hello".match(/(l)(l)/)[1] # => "l"
```

pre_match

text before the match

```
"hello".match(/ll/).pre_match # => "he"
```

post_match

text after the match

```
"hello".match(/ll/).post_match # => "o"
```

to_a

array of whole match plus all groups

```
"hello".match(/(l)(l)/).to_a # => ["ll", "l", "l"]
```

captures

array of capture groups excluding the whole match

```
"hello".match(/(l)(l)/).captures # => ["l", "l"]
```

to_s

the whole matched string

```
"hello".match(/ll/).to_s # => "ll"
```

size

number of groups including group 0 (alias length)

```
"hello".match(/(l)(l)/).size # => 3
```

length

number of groups including group 0 (alias size)

```
"hello".match(/(l)(l)/).length # => 3
```

Array

&

set intersection vs arg array (deduped)

```
[1,2,3] & [2,3,4]  # => [2, 3]
```

intersection

set intersection vs arg array (deduped)

```
[1,2].intersection([2,3])  # => [2]
```

|

set union vs arg array (deduped)

```
[1,2] | [2,3]  # => [1, 2, 3]
```

union

set union vs arg array (deduped)

```
[1,2].union([2,3])  # => [1, 2, 3]
```

-

set difference vs arg array

```
[1,2,3] - [2]  # => [1, 3]
```

difference

set difference vs arg array

```
[1,2,3].difference([2])  # => [1, 3]
```

length

element count

```
[1,2,3].length  # => 3
```

size

element count

```
[1,2].size  # => 2
```

push

append args to end in place, returns self

```
[1].push(2,3) # => [1, 2, 3]
```

append

append args to end in place, returns self

```
[1].append(2) # => [1, 2]
```

<<

append args to end in place, returns self

```
[1] << 2 # => [1, 2]
```

pop

remove and return last element, nil if empty, mutates

```
[1,2].pop # => 2
```

shift

remove and return first element, nil if empty, mutates

```
[1,2].shift # => 1
```

unshift

insert args at front in place, returns self

```
[2].unshift(0,1) # => [0, 1, 2]
```

prepend

insert args at front in place, returns self

```
[3].prepend(1,2) # => [1, 2, 3]
```

first

first element, or first n elements as array if arg given

```
[1,2,3].first # => 1
```

last

last element, or last n elements as array if arg given

```
[1,2,3].last # => 3
```

each_cons

yield each n-wide window; no block returns windows array

```
[1,2,3].each_cons(2) # => [[1, 2], [2, 3]]
```

dig

recursively index into nested arrays/hashes

```
[[1,[2]]].dig(0,1,0) # => 2
```

empty?

true if array has no elements

```
[] .empty? # => true
```

reverse

new array with elements reversed

```
[1,2,3].reverse # => [3, 2, 1]
```

to_a

return a shallow copy of the array

```
[1,2].to_a # => [1, 2]
```

to_ary

return a shallow copy of the array

```
[1,2].to_ary # => [1, 2]
```

dup

return a shallow copy of the array

```
[1,2].dup # => [1, 2]
```

clone

return a shallow copy of the array

```
[1,2].clone # => [1, 2]
```

deconstruct

return a shallow copy of the array

```
[1,2].deconstruct # => [1, 2]
```

include?

true if any element == arg

```
[1,2].include?(2) # => true
```

index

first index where block truthy or element == arg, else nil

```
[1,2,3].index(2) # => 1
```

find_index

first index where block truthy or element == arg, else nil

```
[1,2,3].find_index(3) # => 2
```

rindex

last index where block truthy or element == arg, else nil

```
[1,2,1].rindex(1) # => 2
```

bsearch

binary search sorted array via block; no block returns self

```
[1,2,3,4].bsearch { |x| x >= 3 } # => 3
```

values_at

elements at given indices/ranges, nil for out-of-range

```
[10,20,30].values_at(0,2) # => [10, 30]
```

each_index

yield each index; no block returns indices array

```
[5,6].each_index { |i| } # => [5, 6]
```

rotate!

rotate elements left by n (default 1) in place, returns self

```
[1,2,3].rotate! # => [2, 3, 1]
```

flatten!

flatten nested arrays in place; nil if nothing was nested

```
[1,[2,[3]]].flatten! # => [1, 2, 3]
```

compact!

remove nil elements in place; nil if none removed

```
[1,nil,2].compact! # => [1, 2]
```

join

concatenate elements to string with optional separator

```
[1,2,3].join("-") # => "1-2-3"
```

sort

sort by <=> or block; returns new sorted array

```
[3,1,2].sort # => [1, 2, 3]
```

sort!

sort by <=> or block in place, returns self

```
[3,1,2].sort!    # => [1, 2, 3]
```

minmax

[min, max] pair by <=> or block; [nil, nil] if empty

```
[3,1,2].minmax   # => [1, 3]
```

min

min by <=> or block, or n smallest sorted if arg given

```
[3,1,2].min      # => 1
```

max

max by <=> or block, or n largest sorted if arg given

```
[3,1,2].max      # => 3
```

sum

sum elements (block-mapped) starting from arg init (default 0)

```
[1,2,3].sum      # => 6
```

uniq

new array with duplicates removed (block form unsupported)

```
[1,1,2,2].uniq   # => [1, 2]
```

compact

new array with nil elements removed

```
[1,nil,2].compact # => [1, 2]
```

flatten

new fully-flattened array, or up to n levels if arg given

```
[1,[2,[3]]].flatten # => [1, 2, 3]
```

concat

append elements of arg arrays in place, returns self

```
[1].concat([2,3]) # => [1, 2, 3]
```

each

yield each element, returns self; no block returns Enumerator

```
[1,2].each { |x| } # => [1, 2]
```

each_with_index

yield element and index; no block returns Enumerator of pairs

```
[10,20].each_with_index { |x,i| } # => [10, 20]
```

map

map elements via block; no block returns Enumerator

```
[1,2,3].map { |x| x*2 } # => [2, 4, 6]
```

collect

map elements via block; no block returns Enumerator

```
[1,2].collect { |x| x+1 } # => [2, 3]
```

flat_map

map via block, flattening array results; no block returns Enumerator

```
[[1],[2]].flat_map { |x| x } # => [1, 2]
```

select

keep block-truthy elements; no block returns Enumerator

```
[1,2,3,4].select { |x| x.even? } # => [2, 4]
```

filter

keep block-truthy elements; no block returns Enumerator

```
[1,2,3].filter { |x| x>1 } # => [2, 3]
```

find_all

keep block-truthy elements; no block returns Enumerator

```
[1,2,3].find_all { |x| x>1 } # => [2, 3]
```

reject

drop block-truthy elements; no block returns Enumerator

```
[1,2,3].reject { |x| x>1 } # => [1]
```

filter_map

map elements via block, keep only truthy results

```
[1,2,3].filter_map { |x| x*2 if x.odd? } # => [2, 6]
```

transpose

transpose array of equal-length rows; IndexError if ragged

```
[[1,2],[3,4]].transpose # => [[1, 3], [2, 4]]
```

find

first element where block truthy, else nil

```
[1,2,3].find { |x| x>1 }    # => 2
```

detect

first element where block truthy, else nil

```
[1,2,3].detect { |x| x>2 }    # => 3
```

any?

block truthy for any; block-less: any element truthy

```
[nil,1].any?    # => true
```

all?

block truthy for all elements

```
[1,2].all? { |x| x>0 }    # => true
```

none?

block truthy for no element

```
[1,2].none? { |x| x>5 }    # => true
```

count

count block-truthy elements, or == arg, else total length

```
[1,2,2,3].count(2)    # => 2
```

reduce

fold via block or symbol op with optional initial value

```
[1,2,3,4].reduce(:+)    # => 10
```

inject

fold via block or symbol op with optional initial value

```
[1,2,3].inject(10) { |a,x| a+x }    # => 16
```

min_by

min element by block key

```
["aa","b"].min_by { |s| s.length }    # => "b"
```

max_by

max element by block key

```
["aa","b"].max_by { |s| s.length }    # => "aa"
```

sort_by

sort by block key

```
["bb","a"].sort_by { |s| s.length } # => ["a", "bb"]
```

minmax_by

[min, max] by block key

```
["aa","b"].minmax_by { |s| s.length } # => ["b", "aa"]
```

[]

index or range access

```
[1,2,3][1] # => 2
```

fetch

element at index, else arg default, else block, else IndexError

```
[1,2,3].fetch(1) # => 2
```

[]=

set element at index, padding with nil, returns assigned value

```
a=[1,2,3]; a[1]=9; a # => [1, 9, 3]
```

take

first n elements as new array

```
[1,2,3,4].take(2) # => [1, 2]
```

drop

all but first n elements as new array

```
[1,2,3,4].drop(2) # => [3, 4]
```

partition

[matching, non-matching] arrays split by block

```
[1,2,3,4].partition { |x| x.even? } # => [[2, 4], [1, 3]]
```

group_by

hash grouping elements by block return value

```
[1,2,3,4].group_by { |x| x%2 } # => {1 => [1, 3], 0 => [2, 4]}
```

tally

hash counting occurrences of each element

```
["a","b","a"].tally # => {"a" => 2, "b" => 1}
```

each_with_object

yield element and memo object, returns the memo

```
[1,2,3].each_with_object([]) { |x,a| a << x*2 } # => [2, 4, 6]
```

zip

merge with arg arrays into rows; block yields rows, returns nil

```
[1,2].zip([3,4]) # => [[1, 3], [2, 4]]
```

product

cartesian product of self with arg arrays

```
[1,2].product([3,4]) # => [[1, 3], [1, 4], [2, 3], [2, 4]]
```

combination

n-element combinations; block yields each else returns array

```
[1,2,3].combination(2) # => [[1, 2], [1, 3], [2, 3]]
```

permutation

n-element permutations (default all); block yields each else array

```
[1,2].permutation # => [[1, 2], [2, 1]]
```

assoc

first sub-array whose first element == arg, else nil

```
[[1,"a"],[2,"b"]].assoc(2) # => [2, "b"]
```

rassoc

first sub-array whose second element == arg, else nil

```
[[1,"a"],[2,"b"]].rassoc("b") # => [2, "b"]
```

fill

fill with value or block result over optional range, mutates

```
[1,2,3].fill(0) # => [0, 0, 0]
```

insert

insert args at index (neg inserts after), padding, returns self

```
[1,2,3].insert(1,9) # => [1, 9, 2, 3]
```

delete_at

remove and return element at index, nil if out of range

```
[1,2,3].delete_at(1) # => 2
```

delete_if

remove block-truthy elements in place, returns self

```
[1,2,3,4].delete_if { |x| x.even? } # => [1, 3]
```

reject!

remove block-truthy elements in place, returns self

```
[1,2,3,4].reject! { |x| x.even? } # => [1, 3]
```

take_while

leading elements while block truthy

```
[1,2,3,1].take_while { |x| x<3 } # => [1, 2]
```

drop_while

elements after the leading block-truthy run

```
[1,2,3,1].drop_while { |x| x<3 } # => [3, 1]
```

rotate

new array rotated left by n (default 1)

```
[1,2,3].rotate # => [2, 3, 1]
```

each_slice

yield n-length slices; no block returns slices array

```
[1,2,3,4,5].each_slice(2) # => [[1, 2], [3, 4], [5]]
```

chunk_while

split into runs; new run when block falsey for adjacent pair

```
[1,2,4,5].chunk_while { |a,b| b-a==1 } # => [[1, 2], [4, 5]]
```

slice_when

split into runs; new run when block truthy for adjacent pair

```
[1,2,4,5].slice_when { |a,b| b-a>1 } # => [[1, 2], [4, 5]]
```

to_h

build hash from [k,v] pairs, block maps each element first

```
[[1,2],[3,4]].to_h # => {1 => 2, 3 => 4}
```

cycle

repeat elements n times or endlessly to block

```
[1,2].cycle(2) { |x| } # => nil
```

chunk

consecutive equal-key runs as [key, elems] pairs

```
[1,1,2].chunk { |x| x } # => [[1, [1, 1]], [2, [2]]]
```

Hash

size

number of key-value pairs

```
{a: 1, b: 2}.size # => 2
```

length

number of key-value pairs

```
{a: 1}.length # => 1
```

count

count block-truthy pairs, or total pairs without block

```
{a: 1, b: 2}.count # => 2
```

empty?

true if hash has no pairs

```
{}.empty? # => true
```

deconstruct_keys

return self for pattern matching (keys arg is only a hint)

```
{a: 1}.deconstruct_keys(nil) # => {a: 1}
```

keys

array of keys

```
{a: 1, b: 2}.keys # => [:a, :b]
```

values

array of values

```
{a: 1, b: 2}.values # => [1, 2]
```

key?

true if hash has the given key

```
{a: 1}.key?(:a) # => true
```

has_key?

true if hash has the given key

```
{a: 1}.has_key?(:a)  # => true
```

include?

true if hash has the given key

```
{a: 1}.include?(:a)  # => true
```

member?

true if hash has the given key

```
{a: 1}.member?(:a)  # => true
```

value?

true if any value == arg

```
{a: 1}.value?(1)  # => true
```

has_value?

true if any value == arg

```
{a: 1}.has_value?(1)  # => true
```

[]

value for key, else default proc call, else default value (nil)

```
{a: 1}[:a]  # => 1
```

fetch

value for key, else block, else arg default, else KeyError

```
{a: 1}.fetch(:a)  # => 1
```

[]=

set value for key, returns the value

```
h={}; h[:a]=1; h  # => {a: 1}
```

store

set value for key, returns the value

```
h={}; h.store(:a,1); h  # => {a: 1}
```

delete

remove key and return its value, nil if absent

```
{a: 1, b: 2}.delete(:a)  # => 1
```

merge

new hash merging arg hashes; block resolves key collisions

```
{a: 1}.merge({b: 2}) # => {a: 1, b: 2}
```

to_a

array of [key, value] pairs

```
{a: 1, b: 2}.to_a # => [[:a, 1], [:b, 2]]
```

each

yield key and value, returns self

```
{a: 1}.each { |k,v| } # => {a: 1}
```

each_pair

yield key and value, returns self

```
{a: 1}.each_pair { |k,v| } # => {a: 1}
```

map

array of block results, block gets each [k, v] pair

```
{a: 1, b: 2}.map { |k,v| v*2 } # => [2, 4]
```

select

new hash keeping block-truthy pairs

```
{a: 1, b: 2}.select { |k,v| v>1 } # => {b: 2}
```

filter

new hash keeping block-truthy pairs

```
{a: 1, b: 2}.filter { |k,v| v>1 } # => {b: 2}
```

reject

new hash dropping block-truthy pairs

```
{a: 1, b: 2}.reject { |k,v| v>1 } # => {a: 1}
```

transform_values

new hash with each value replaced by block result

```
{a: 1, b: 2}.transform_values { |v| v*10 } # => {a: 10, b: 20}
```

transform_keys

new hash with keys remapped via mapping hash then block

```
{a: 1}.transform_keys { |k| k.to_s } # => {"a" => 1}
```

filter_map

array of truthy block results over pairs

```
{a: 1, b: 2}.filter_map { |k,v| k if v>1 } # => [:b]
```

each_with_object

yield [k, v] pair and memo, returns the memo

```
{a: 1, b: 2}.each_with_object([]) { |(k,v),a| a << k } # => [:a, :b]
```

sum

sum of block results over pairs from arg init (default 0)

```
{a: 1, b: 2}.sum { |k,v| v } # => 3
```

any?

block truthy for any pair; without block returns false

```
{a: 1}.any? # => false
```

all?

block truthy for all pairs; without block returns true

```
{}.all? # => true
```

none?

block truthy for no pair; without block returns true

```
{}.none? # => true
```

min_by

min [k, v] pair by block key

```
{a: 2, b: 1}.min_by { |k,v| v } # => [:b, 1]
```

max_by

max [k, v] pair by block key

```
{a: 2, b: 1}.max_by { |k,v| v } # => [:a, 2]
```

sort_by

sort [k, v] pairs by block key

```
{a: 2, b: 1}.sort_by { |k,v| v } # => [[:b, 1], [:a, 2]]
```

reduce

fold over [k, v] pairs

```
{a: 1, b: 2}.reduce(0) { |s,(k,v)| s+v } # => 3
```

inject

fold over [k, v] pairs

```
{a: 1, b: 2}.inject(0) { |s,(k,v)| s+v } # => 3
```

group_by

group [k, v] pairs by block return

```
{a: 1, b: 2}.group_by { |k,v| v.odd? } # => {true => [[:a, 1]], false => [[:b, 2]]}
```

partition

partition [k, v] pairs by block

```
{a: 1, b: 2}.partition { |k,v| v>1 } # => [[[:b, 2]], [[:a, 1]]]
```

find_all

keep block-truthy [k, v] pairs

```
{a: 1, b: 2}.find_all { |k,v| v>1 } # => [[:b, 2]]
```

invert

new hash with keys and values swapped

```
{a: 1, b: 2}.invert # => {1 => :a, 2 => :b}
```

default

the hash's default value for missing keys

```
Hash.new(0).default # => 0
```

default=

set the hash's default value, returns it

```
h={}; h.default=5; h[:x] # => 5
```

to_h

return self

```
{a: 1}.to_h # => {a: 1}
```

except

new hash without the given keys, order preserved

```
{a: 1, b: 2, c: 3}.except(:b) # => {a: 1, c: 3}
```

slice

new hash with only the given keys, in argument order

```
{a: 1, b: 2, c: 3}.slice(:a,:c) # => {a: 1, c: 3}
```

compact

new hash without nil-valued pairs

```
{a: 1, b: nil}.compact # => {a: 1}
```

flat_map

map pairs via block, flattening array results

```
{a: 1, b: 2}.flat_map { |k,v| [k,v] } # => [:a, 1, :b, 2]
```

collect_concat

map pairs via block, flattening array results

```
{a: 1}.collect_concat { |k,v| [k,v] } # => [:a, 1]
```

each_with_index

yield [k,v] pair and index; no block returns pairs array

```
{a: 1, b: 2}.each_with_index { |pair,i| } # => {a: 1, b: 2}
```

find

first [k, v] pair where block truthy, else nil

```
{a: 1, b: 2}.find { |k,v| v>1 } # => [:b, 2]
```

detect

first [k, v] pair where block truthy, else nil

```
{a: 1, b: 2}.detect { |k,v| v>1 } # => [:b, 2]
```

dig

recursively index into nested hashes/arrays

```
{a: {b: 1}}.dig(:a,:b) # => 1
```

Set

add

add element to set, returns self

```
Set[1].add(2)  # => Set[1, 2]
```

<<

add element to set, returns self

```
Set[1] << 2  # => Set[1, 2]
```

add?

add element; self if newly added, nil if already present

```
Set[1].add?(1)  # => nil
```

delete

remove element from set, returns self

```
Set[1,2].delete(1)  # => Set[2]
```

delete?

remove element; self if it was present, else nil

```
Set[1].delete?(9)  # => nil
```

include?

true if set contains arg

```
Set[1,2].include?(1)  # => true
```

member?

true if set contains arg

```
Set[1].member?(1)  # => true
```

===

true if set contains arg

```
Set[1,2].include?(1)  # => true
```

contain?

true if set contains arg

```
Set[1].contain?(1)  # => true
```

size

number of elements

```
Set[1,2,3].size  # => 3
```

length

number of elements

```
Set[1,2].length  # => 2
```

count

number of elements

```
Set[1,2].count  # => 2
```

empty?

true if set has no elements

```
Set[].empty?  # => true
```

clear

remove all elements, returns self

```
Set[1,2].clear  # => Set[]
```

to_a

elements as a new array

```
Set[1,2].to_a  # => [1, 2]
```

to_ary

elements as a new array

```
Set[1,2].to_ary  # => [1, 2]
```

to_set

return a copy as a new set

```
Set[1].to_set  # => Set[1]
```

dup

return a copy as a new set

```
Set[1].dup  # => Set[1]
```

clone

return a copy as a new set

```
Set[1].clone # => Set[1]
```

merge

add all elements of arg set/array in place, returns self

```
Set[1].merge([2,3]) # => Set[1, 2, 3]
```

union

new set with elements of self and arg

```
Set[1].union(Set[2]) # => Set[1, 2]
```

|

new set with elements of self and arg

```
Set[1] | Set[2] # => Set[1, 2]
```

+

new set with elements of self and arg

```
Set[1] + Set[2] # => Set[1, 2]
```

merge_new

new set with elements of self and arg

```
Set[1].merge_new(Set[2]) # => Set[1, 2]
```

intersection

new set of elements in both self and arg

```
Set[1,2].intersection(Set[2,3]) # => Set[2]
```

&

new set of elements in both self and arg

```
Set[1,2] & Set[2,3] # => Set[2]
```

difference

new set of elements in self but not arg

```
Set[1,2].difference(Set[2]) # => Set[1]
```

-

new set of elements in self but not arg

```
Set[1,2] - Set[2] # => Set[1]
```

^

new set of symmetric difference (elements in exactly one)

```
Set[1,2] ^ Set[2,3]  # => Set[1, 3]
```

subset?

true if every element of self is in arg

```
Set[1].subset?(Set[1,2])  # => true
```

<=

true if every element of self is in arg

```
Set[1].subset?(Set[1,2])  # => true
```

superset?

true if every element of arg is in self

```
Set[1,2].superset?(Set[1])  # => true
```

>=

true if every element of arg is in self

```
Set[1,2].superset?(Set[1])  # => true
```

proper_subset?

true if self is subset of arg and strictly smaller

```
Set[1].proper_subset?(Set[1,2])  # => true
```

<

true if self is subset of arg and strictly smaller

```
Set[1].proper_subset?(Set[1,2])  # => true
```

proper_superset?

true if self is superset of arg and strictly larger

```
Set[1,2].proper_superset?(Set[1])  # => true
```

>

true if self is superset of arg and strictly larger

```
Set[1,2].proper_superset?(Set[1])  # => true
```

disjoint?

true if self and arg share no elements

```
Set[1].disjoint?(Set[2])  # => true
```

intersect?

true if self and arg share any element

```
Set[1,2].intersect?(Set[2,3]) # => true
```

each

yield each element, returns self (block required)

```
Set[1,2].each { |x| } # => Set[1, 2]
```

Range

to_a

materialize integer range into Array

```
(1..3).to_a # => [1, 2, 3]
```

to_ary

materialize integer range into Array

```
(1..3).to_ary # => [1, 2, 3]
```

entries

materialize integer range into Array

```
(1..3).entries # => [1, 2, 3]
```

first

lower bound, or first n elements as Array with count arg

```
(1..5).first # => 1
```

last

upper bound, or last n elements as Array with count arg

```
(1..5).last # => 5
```

min

lower bound lo

```
(1..5).min # => 1
```

begin

lower bound lo

```
(1..5).begin # => 1
```

max

upper bound (hi-1 if exclusive)

```
(1...5).max # => 4
```

end

raw upper bound hi (not exclusive-adjusted)

```
(1..5).end    # => 5
```

size

element count (end-lo, min 0)

```
(1..5).size    # => 5
```

count

element count (end-lo, min 0)

```
(1..5).count    # => 5
```

length

element count (end-lo, min 0)

```
(1..5).length    # => 5
```

sum

sum of integer elements

```
(1..5).sum    # => 15
```

include?

true if arg in [lo, end)

```
(1..5).include?(3)    # => true
```

cover?

true if arg in [lo, end)

```
(1..5).cover?(5)    # => true
```

member?

true if arg in [lo, end)

```
(1..5).member?(3)    # => true
```

===

true if arg in [lo, end)

```
(1..5) === 5    # => true
```

step

step by Int n or Float; yields to block or returns Array/enum

```
(1..10).step(3).to_a    # => [1, 4, 7, 10]
```

each

yield each integer to block

```
(1..3).each { |x| } # => 1..3
```

exclude_end?

true if range excludes its end

```
(1.0...2.0).exclude_end? # => true
```

%

alias of step: float step by arg

```
((1.0..3.0) % 0.5).to_a # => [1.0, 1.5, 2.0, 2.5, 3.0]
```

Struct

to_a

array of member values

```
S=Struct.new(:a,:b); S.new(1,2).to_a # => [1, 2]
```

values

array of member values

```
S=Struct.new(:a,:b); S.new(1,2).values # => [1, 2]
```

deconstruct

array of member values (for array pattern matching)

```
S=Struct.new(:a,:b); S.new(1,2).deconstruct # => [1, 2]
```

members

member names as symbols

```
S=Struct.new(:a,:b); S.new(1,2).members # => [:a, :b]
```

size

number of members

```
S=Struct.new(:a,:b); S.new(1,2).size # => 2
```

length

number of members

```
S=Struct.new(:a,:b); S.new(1,2).length # => 2
```

to_h

hash of member => value

```
S=Struct.new(:a,:b); S.new(1,2).to_h # => {a: 1, b: 2}
```

deconstruct_keys

hash of requested (or all) members for pattern matching

```
S=Struct.new(:a); S.new(1).deconstruct_keys(nil) # => {a: 1}
```

[]

index by int position or by symbol/string member name

```
S=Struct.new(:a,:b); S.new(1,2)[:b] # => 2
```

each

yields each member value, returns receiver (block form only)

```
S=Struct.new(:a); S.new(1).each { |v| v } # => #<struct S a=1>
```

each_pair

yields member symbol and value, returns receiver (block form)

```
S=Struct.new(:a); S.new(1).each_pair { |m,v| } # => #<struct S a=1>
```

==

true if same struct class and all members equal

```
S=Struct.new(:a); S.new(1) == S.new(1) # => true
```

eql?

true if same struct class and all members equal (same as ==)

```
S=Struct.new(:a); S.new(1).eql?(S.new(1)) # => true
```

to_s

#<struct Name m=v, ...> representation

```
S=Struct.new(:a); S.new(1).to_s # => "#<struct S a=1>"
```

inspect

#<struct Name m=v, ...> representation

```
S=Struct.new(:a); S.new(1).inspect # => "#<struct S a=1>"
```

Exception

message

the message ivar, or the class name if unset

```
begin; raise "boom"; rescue => e; e.message; end # => "boom"
```

to_s

the message ivar, or the class name if unset

```
begin; raise "boom"; rescue => e; e.to_s; end # => "boom"
```

Enumerator

next

Returns next buffered value; Stop iteration at end.

```
e=[1,2,3].each; e.next    # => 1
```

peek

Returns next buffered value without advancing; Stop iteration at end.

```
e=[1,2,3].each; e.peek    # => 1
```

rewind

Resets the buffer cursor; returns self.

```
e=[1,2,3].each; e.next; e.rewind; e.next    # => 1
```

size

Returns buffer length as Int (nil for generator source).

```
[1,2,3].each.size    # => 3
```

length

Returns buffer length as Int.

```
[1,2,3].each.length    # => 3
```

with_index

Re-runs source method with index; returns elements array.

```
[10,20].each.with_index { |x,i| [x,i] }    # => [10, 20]
```

with_object

Threads memo through block per element; returns memo.

```
[1,2].each.with_object([]) { |x,a| a<<x }    # => [1, 2]
```

each_with_object

Threads memo through block per element; returns memo.

```
[1,2].each.each_with_object([]) { |x,a| a<<x }    # => [1, 2]
```

first

Drives source for first value, or n as array.

```
[1,2,3].each.first(2)  # => [1, 2]
```

take

Drives source collecting up to n values as array.

```
[1,2,3].each.take(2)  # => [1, 2]
```

to_a

Drives source to completion into array.

```
[1,2,3].each.to_a  # => [1, 2, 3]
```

force

Drives source to completion into array.

```
[1,2,3].each.force  # => [1, 2, 3]
```

entries

Drives source to completion into array.

```
[1,2,3].each.entries  # => [1, 2, 3]
```

each

Drives source to completion, calls block per value; returns self.

```
s=0; [1,2,3].each.each { |x| s+=x }; s  # => 6
```

lazy

Wraps the enumerator as a lazy pipeline source.

```
[1,2,3].each.lazy.first(2)  # => [1, 2]
```

Enumerator::Lazy

map

Appends a lazy Map op; returns new lazy enumerator.

```
(1..Float::INFINITY).lazy.map { |x| x*2 }.first(3) # => [2, 4, 6]
```

collect

Appends a lazy Map op; returns new lazy enumerator.

```
(1..Float::INFINITY).lazy.collect { |x| x+1 }.first(2) # => [2, 3]
```

select

Appends a lazy Select op; returns new lazy enumerator.

```
(1..Float::INFINITY).lazy.select { |x| x.even? }.first(2) # => [2, 4]
```

filter

Appends a lazy Select op; returns new lazy enumerator.

```
(1..Float::INFINITY).lazy.filter { |x| x.even? }.first(2) # => [2, 4]
```

reject

Appends a lazy Reject op; returns new lazy enumerator.

```
[1,2,3,4].lazy.reject { |x| x.even? }.to_a # => [1, 3]
```

filter_map

Appends a lazy FilterMap op; returns new lazy enumerator.

```
[1,2,3].lazy.filter_map { |x| x*2 if x.odd? }.to_a # => [2, 6]
```

flat_map

Appends a lazy FlatMap op; returns new lazy enumerator.

```
[1,2,3].lazy.flat_map { |x| [x,x] }.first(4) # => [1, 1, 2, 2]
```

collect_concat

Appends a lazy FlatMap op; returns new lazy enumerator.

```
[1,2].lazy.collect_concat { |x| [x,-x] }.to_a # => [1, -1, 2, -2]
```

take_while

Appends a lazy TakeWhile op; returns new lazy enumerator.

```
(1..9).lazy.take_while { |x| x < 4 }.to_a  # => [1, 2, 3]
```

drop_while

Appends a lazy DropWhile op; returns new lazy enumerator.

```
(1..5).lazy.drop_while { |x| x < 3 }.to_a  # => [3, 4, 5]
```

take

Appends a lazy Take(n) op; returns new lazy enumerator.

```
(1..5).lazy.take(2).to_a  # => [1, 2]
```

drop

Appends a lazy Drop(n) op; returns new lazy enumerator.

```
(1..5).lazy.drop(2).to_a  # => [3, 4, 5]
```

zip

Appends a lazy Zip op over array-coerced args.

```
[1,2].lazy.zip([3,4]).to_a  # => [[1, 3], [2, 4]]
```

lazy

Returns self unchanged.

```
[1,2,3].lazy.lazy.first(2)  # => [1, 2]
```

first

Pulls first element, or first n as array if arg given.

```
(1..Float::INFINITY).lazy.first(3)  # => [1, 2, 3]
```

force

Pulls the whole pipeline into an array.

```
[1,2,3].lazy.map { |x| x }.force  # => [1, 2, 3]
```

to_a

Pulls the whole pipeline into an array.

```
[1,2,3].lazy.map { |x| x }.to_a  # => [1, 2, 3]
```

entries

Pulls the whole pipeline into an array.

```
[1,2,3].lazy.map { |x| x }.entries  # => [1, 2, 3]
```

each

With block: pulls all and calls block per element; returns self.

```
s=0; [1,2,3].lazy.each { |x| s+=x }; s # => 6
```

Enumerator::Yielder

<<

Pushes value into sink; raises break at limit; returns self.

```
Enumerator.new { |y| y << 1 }.to_a # => [1]
```

yield

Pushes value into sink; raises break at limit; returns self.

```
Enumerator.new { |y| y.yield 2 }.to_a # => [2]
```

Fiber

resume

Resumes the fiber with args (nil/one/array).

```
f=Fiber.new { Fiber.yield 5 }; f.resume # => 5
```

alive?

True if the fiber is still alive.

```
f=Fiber.new { Fiber.yield 5 }; f.resume; f.alive? # => true
```

inspect

Returns "#<Fiber (created)>" with " (dead)" if not alive.

```
Fiber.new {}.inspect # => "#<Fiber (created)>"
```

to_s

Returns "#<Fiber (created)>" with " (dead)" if not alive.

```
Fiber.new {}.to_s # => "#<Fiber (created)>"
```

Proc

call

Symbol-proc sends the symbol to args[0]; else invokes the proc.

```
d-->(x){ x*2 }; d.call(3) # => 6
```

[]

Symbol-proc sends the symbol to args[0]; else invokes the proc.

```
d-->(x){ x*2 }; d[3] # => 6
```

yield

Symbol-proc sends the symbol to args[0]; else invokes the proc.

```
d-->(x){ x*2 }; d.yield(3) # => 6
```

to_proc

Returns self unchanged.

```
d-->(x){ x }; d.to_proc.call(5) # => 5
```

arity

Returns the proc's arity as Int (0 if unknown).

```
->(a,b){ a+b }.arity # => 2
```

lambda?

True if the proc is a lambda.

```
->(x){ x }.lambda? # => true
```

curry

Returns a curried version of the proc.

```
c-->(a,b){ a+b }; c.curry[1][2] # => 3
```

>>

Composition: $(f \gg g).call(x) == g.call(f.call(x))$.

```
(->(x){ x+1 }) >> (->(x){ x*2 }).call(3) # => 8
```

<<

Composition: $(f \ll g).call(x) == f.call(g.call(x))$.

```
(->(x){ x+1 }) << ->(x){ x*2 }).call(3)  # => 7
```

Method

call

Routes back through dispatch on the captured receiver.

```
"hi".method(:upcase).call # => "HI"
```

[]

Routes back through dispatch on the captured receiver.

```
m="hi".method(:upcase); m[] # => "HI"
```

yield

Routes back through dispatch on the captured receiver.

```
m="hi".method(:upcase); m.yield # => "HI"
```

===

Routes back through dispatch on the captured receiver.

```
m=4.method(:even?); m.call # => true
```

arity

Returns the bound method's arity as Int.

```
"hi".method(:upcase).arity # => -1
```

name

Returns the method name as a Symbol.

```
"hi".method(:upcase).name # => :upcase
```

receiver

Returns the captured receiver.

```
"hi".method(:upcase).receiver # => "hi"
```

to_proc

Returns self unchanged (Method is callable).

```
"hi".method(:upcase).to_proc.call # => "HI"
```

Time

year

year field (UTC breakdown of epoch secs)

```
Time.at(0).year    # => 1970
```

month

month 1-12

```
Time.at(90061).month  # => 1
```

mon

month 1-12

```
Time.at(90061).mon    # => 1
```

day

day of month

```
Time.at(90061).day    # => 2
```

mday

day of month

```
Time.at(90061).mday   # => 2
```

hour

hour 0-23

```
Time.at(90061).hour   # => 1
```

min

minute 0-59

```
Time.at(90061).min    # => 1
```

sec

second 0-59

```
Time.at(90061).sec    # => 1
```

wday

weekday, 0=Sunday

```
Time.at(0).wday # => 4
```

yday

day of year

```
Time.at(90061).yday # => 2
```

to_i

epoch seconds floored to Integer

```
Time.at(90061).to_i # => 90061
```

tv_sec

epoch seconds floored to Integer

```
Time.at(90061).tv_sec # => 90061
```

to_f

epoch seconds as Float

```
Time.at(90061).to_f # => 90061.0
```

sunday?

true if wday == 0

```
Time.at(0).sunday? # => false
```

monday?

true if wday == 1

```
Time.at(0).monday? # => false
```

tuesday?

true if wday == 2

```
Time.at(0).tuesday? # => false
```

wednesday?

true if wday == 3

```
Time.at(0).wednesday? # => false
```

thursday?

true if wday == 4

```
Time.at(0).thursday? # => true
```

friday?

true if wday == 5

```
Time.at(90061).friday?  # => true
```

saturday?

true if wday == 6

```
Time.at(0).saturday?  # => false
```

utc

no-op: returns equivalent Time (UTC-only, no tz offset)

```
Time.at(0).utc.year  # => 1970
```

getutc

no-op: returns equivalent Time (UTC-only, no tz offset)

```
Time.at(0).getutc.year  # => 1970
```

gmtime

no-op: returns equivalent Time (UTC-only, no tz offset)

```
Time.at(0).gmtime.year  # => 1970
```

localtime

no-op: returns equivalent Time (UTC-only, no tz offset)

```
Time.at(0).localtime.year  # => 1970
```

getlocal

no-op: returns equivalent Time (UTC-only, no tz offset)

```
Time.at(0).getlocal.year  # => 1970
```

utc?

always true (UTC-only model)

```
Time.at(0).utc?  # => true
```

gmt?

always true (UTC-only model)

```
Time.at(0).gmt?  # => true
```

to_s

formatted time string

```
Time.at(0).to_s  # => "1970-01-01 00:00:00 UTC"
```

inspect

formatted time string (inspect form)

```
Time.at(0).inspect    # => "1970-01-01 00:00:00 UTC"
```

strftime

format time by strftime pattern arg

```
Time.at(0).strftime("%Y")    # => "1970"
```

<=>

compare epoch secs; nil if arg not a Time

```
Time.at(0) <=> Time.at(1)    # => -1
```

==

true if arg has equal epoch secs

```
Time.at(0) == Time.at(0)    # => true
```

+

new Time shifted forward by arg seconds

```
(Time.at(0) + 86400).day    # => 2
```

-

Float secs diff if arg is Time, else Time minus arg secs

```
Time.at(10) - Time.at(0)    # => 10.0
```

hash

epoch secs bit pattern as Integer

```
Time.at(0).hash    # => 0
```

Date

year

year field

```
Date.new(2024,3,4).year    # => 2024
```

month

month 1-12

```
Date.new(2024,3,4).month  # => 3
```

mon

month 1-12

```
Date.new(2024,3,4).mon    # => 3
```

day

day of month

```
Date.new(2024,3,4).day    # => 4
```

mday

day of month

```
Date.new(2024,3,4).mday   # => 4
```

wday

weekday, 0=Sunday

```
Date.new(2024,3,4).wdays  # => 1
```

yday

day of year

```
Date.new(2024,3,4).yday   # => 64
```

cwday

ISO weekday, Monday=1..Sunday=7

```
Date.new(2024,3,4).cwday  # => 1
```

jd

Julian day number

```
Date.new(2024,1,15).jd    # => 2460325
```

leap?

true if year is a leap year

```
Date.new(2024,1,1).leap?  # => true
```

sunday?

true if wday == 0

```
Date.new(2024,3,4).sunday?  # => false
```

monday?

true if wday == 1

```
Date.new(2024,3,4).monday?  # => true
```

tuesday?

true if wday == 2

```
Date.new(2024,3,4).tuesday?  # => false
```

wednesday?

true if wday == 3

```
Date.new(2024,3,4).wednesday?  # => false
```

thursday?

true if wday == 4

```
Date.new(2024,3,4).thursday?  # => false
```

friday?

true if wday == 5

```
Date.new(2024,3,4).friday?  # => false
```

saturday?

true if wday == 6

```
Date.new(2024,3,4).saturday?  # => false
```

to_s

ISO date string YYYY-MM-DD

```
Date.new(2024,1,15).to_s    # => "2024-01-15"
```

iso8601

ISO date string YYYY-MM-DD

```
Date.new(2024,1,15).iso8601 # => "2024-01-15"
```

inspect

inspect string form

```
Date.new(2024,1,15).inspect # => "#<Date: 2024-01-15>"
```

strftime

format date by strftime pattern arg

```
Date.new(2024,1,15).strftime("%Y") # => "2024"
```

next_day

date plus arg days (default 1)

```
Date.new(2024,1,1).next_day.to_s # => "2024-01-02"
```

succ

date plus arg days (default 1)

```
Date.new(2024,1,1).succ.to_s # => "2024-01-02"
```

prev_day

date minus arg days (default 1)

```
Date.new(2024,1,2).prev_day.to_s # => "2024-01-01"
```

next_month

add arg months, clamping day to month end (default 1)

```
Date.new(2024,1,31).next_month.to_s # => "2024-02-29"
```

prev_month

subtract arg months, clamping day to month end (default 1)

```
Date.new(2024,3,31).prev_month.to_s # => "2024-02-29"
```

>>

add arg months, clamping day to month end

```
(Date.new(2024,1,31) >> 1).to_s # => "2024-02-29"
```

<<

subtract arg months, clamping day to month end

```
(Date.new(2024,3,31) << 1).to_s # => "2024-02-29"
```

next_year

add 12*arg months (default 1 year)

```
Date.new(2024,1,1).next_year.to_s # => "2025-01-01"
```

prev_year

subtract 12*arg months (default 1 year)

```
Date.new(2024,1,1).prev_year.to_s # => "2023-01-01"
```

+

date plus arg days

```
(Date.new(2024,1,1) + 1).to_s # => "2024-01-02"
```

-

Rational day diff if arg is Date, else date minus arg days

```
Date.new(2024,1,2) - Date.new(2024,1,1) # => (1/1)
```

<=>

compare day counts; nil if arg not a Date

```
Date.new(2024,1,1) <=> Date.new(2024,1,2) # => -1
```

==

true if arg is the same date

```
Date.new(2024,1,1) == Date.new(2024,1,1) # => true
```

hash

day count as Integer

```
Date.new(1970,1,1).hash # => 0
```

DateTime

year

year field

```
DateTime.new(2024,3,4,9,8,7).year    # => 2024
```

month

month 1-12

```
DateTime.new(2024,3,4,9,8,7).month  # => 3
```

mon

month 1-12

```
DateTime.new(2024,3,4,9,8,7).mon    # => 3
```

day

day of month

```
DateTime.new(2024,3,4,9,8,7).day    # => 4
```

mday

day of month

```
DateTime.new(2024,3,4,9,8,7).mday   # => 4
```

hour

hour 0-23

```
DateTime.new(2024,3,4,9,8,7).hour   # => 9
```

min

minute 0-59

```
DateTime.new(2024,3,4,9,8,7).min    # => 8
```

sec

second 0-59

```
DateTime.new(2024,3,4,9,8,7).sec    # => 7
```

wday

weekday, 0=Sunday

```
DateTime.new(2024,3,4,9,8,7).wday    # => 1
```

yday

day of year

```
DateTime.new(2024,3,4,9,8,7).yday    # => 64
```

cwday

ISO weekday, Monday=1..Sunday=7

```
DateTime.new(2024,3,4,9,8,7).cwday    # => 1
```

jd

Julian day number of the day part

```
DateTime.new(2024,3,4,9,8,7).jd    # => 2460374
```

leap?

true if year is a leap year

```
DateTime.new(2024,1,1,0,0,0).leap?    # => true
```

sunday?

true if wday == 0

```
DateTime.new(2024,3,4,9,8,7).sunday?    # => false
```

monday?

true if wday == 1

```
DateTime.new(2024,3,4,9,8,7).monday?    # => true
```

tuesday?

true if wday == 2

```
DateTime.new(2024,3,4,9,8,7).tuesday?    # => false
```

wednesday?

true if wday == 3

```
DateTime.new(2024,3,4,9,8,7).wednesday?    # => false
```

thursday?

true if wday == 4

```
DateTime.new(2024,3,4,9,8,7).thursday?    # => false
```

friday?

true if wday == 5

```
DateTime.new(2024,3,4,9,8,7).friday? # => false
```

saturday?

true if wday == 6

```
DateTime.new(2024,3,4,9,8,7).saturday? # => false
```

to_s

ISO8601 datetime string

```
DateTime.new(2024,1,15,12,30,0).to_s # => "2024-01-15T12:30:00+00:00"
```

iso8601

ISO8601 datetime string

```
DateTime.new(2024,1,15,12,30,0).iso8601 # => "2024-01-15T12:30:00+00:00"
```

inspect

inspect string form

```
DateTime.new(2024,1,15,0,0,0).inspect # => "#<DateTime: 2024-01-15T00:00:00+00:00>"
```

strftime

format datetime by strftime pattern arg

```
DateTime.new(2024,3,4,9,8,7).strftime("%Y") # => "2024"
```

to_date

Date of the day part

```
DateTime.new(2024,3,4,9,8,7).to_date.to_s # => "2024-03-04"
```

to_time

Time at the same epoch secs

```
DateTime.new(2024,3,4,9,8,7).to_time.to_s # => "2024-03-04 09:08:07 UTC"
```

next_day

datetime plus arg days, keeping time (default 1)

```
DateTime.new(2024,1,1,5,0,0).next_day.to_s # => "2024-01-02T05:00:00+00:00"
```

succ

datetime plus arg days, keeping time (default 1)

```
DateTime.new(2024,1,1,5,0,0).succ.to_s # => "2024-01-02T05:00:00+00:00"
```

prev_day

datetime minus arg days, keeping time (default 1)

```
DateTime.new(2024,1,2,5,0,0).prev_day.to_s # => "2024-01-01T05:00:00+00:00"
```

next_month

add arg months, clamping day, keeping time (default 1)

```
DateTime.new(2024,1,31,5,0,0).next_month.to_s # => "2024-02-29T05:00:00+00:00"
```

>>

add arg months, clamping day, keeping time of day

```
(DateTime.new(2024,1,31,5,0,0) >> 1).to_s # => "2024-02-29T05:00:00+00:00"
```

prev_month

subtract arg months, clamping day, keeping time (default 1)

```
DateTime.new(2024,3,31,5,0,0).prev_month.to_s # => "2024-02-29T05:00:00+00:00"
```

<<

subtract arg months, clamping day, keeping time of day

```
(DateTime.new(2024,3,31,5,0,0) << 1).to_s # => "2024-02-29T05:00:00+00:00"
```

next_year

add 12*arg months, keeping time (default 1 year)

```
DateTime.new(2024,1,1,5,0,0).next_year.to_s # => "2025-01-01T05:00:00+00:00"
```

prev_year

subtract 12*arg months, keeping time (default 1 year)

```
DateTime.new(2024,1,1,5,0,0).prev_year.to_s # => "2023-01-01T05:00:00+00:00"
```

+

new DateTime plus arg days

```
(DateTime.new(2024,1,1,0,0,0) + 1).day # => 2
```

-

Rational day diff if arg is DateTime, else minus arg days

```
DateTime.new(2024,1,2,0,0,0) - DateTime.new(2024,1,1,0,0,0) # => (1/1)
```

<=>

compare epoch secs; nil if arg not a DateTime

```
DateTime.new(2024,1,1,0,0,0) <=> DateTime.new(2024,1,2,0,0,0) # => -1
```

`==`

true if arg has equal epoch secs

```
DateTime.new(2024,1,1,0,0,0) == DateTime.new(2024,1,1,0,0,0) # => true
```

`hash`

epoch secs bit pattern as Integer

```
DateTime.new(1970,1,1,0,0,0).hash # => 0
```